ITHICA: Intra-Thread Instruction Checking Approach for Defect-Induced Silent Data Corruptions

MICRO 2026 Submission **#1700** – Confidential Draft – Do NOT Distribute!!

1 Supplemental Material

1.1 ITHICA LLVM Implementation

We implement each of the four ITHICA transformations, as well as their combinations (`Arith+Mem`, `Arith+MemDiv`, `Arith+MemDiv+Br`) as distinct LLVM IR compiler passes. Each pass can be configured with different block size and interleaving values via command-line options. The resulting shared object file (`.so`) integrates directly into Clang-14's built-in compiler pipeline. Libraries managed by the build system are automatically compiled with our passes, while those explicitly linked are manually compiled with our passes. Implementing ITHICA at the LLVM IR level provides portability, as IR passes are target-agnostic and can be integrated into existing build systems without requiring modifications to the compiler.

Correct instrumentation requires validation instructions to be side-effect free (to preserve program behavior) and that original and validation instructions yield the same output in the absence of errors. All LLVM instructions instrumented by `Arith` satisfy these properties, and can therefore be safely duplicated. For LLVM intrinsics and inline assembly, also instrumented by `Arith`, we verify via attributes that they are side-effect free before duplication. `Br` inserts a validation stack store and load with no side-effects.

For `Mem` and `MemDiv`, we exclude atomic/volatile loads and stores due to their memory side-effects. Other instructions that are excluded for the same reason are *alloca*, *atomicrmw*, and *cmpexchange* [9]; *fence* does not have an architectural effect to check. Additionally, even non-atomic/volatile loads and stores can exhibit data races in thread-unsafe multithreaded code. OpenSSL intentionally permits certain data races [13], so we do not instrument it with `Mem` and `MemDiv`. Despite these restrictions that limit coverage (i.e., fraction of checked instructions), `Arith` *and* `Br` *safely instrument all code*, and *thread-safe code is safely instrumented by all passes.*

Finally, ITHICA also excludes `void` instructions (except for stores) and structural instructions whose duplication would affect the correct SSA form of LLVM: *phi*, *landingpad*, *catchpad*, *cleanuppad*.

Table 2 shows the number of LLVM opcodes and static instructions (program counters, PCs) covered by each ITHICA pass. Combined, ITHICA passes instrument a total of 84% of opcodes and 84.8% of all static instructions in cpu-check, among the ones with checkable architectural effects.

We apply ITHICA as the final IR-level pass, to minimize interference with compiler optimization passes. Despite that, backend compiler optimizations can still eliminate our inserted instructions. To prevent this without modifying compiler internals, we mark duplicated loads and stores as volatile. For arithmetic instructions, we pass their arguments through no-op volatile store-load pairs to achieve the same effect. Future versions of Clang may offer intrinsics to preserve side-effect-free instructions more cleanly.

ITHICA instrumentation can affect how the program's instructions are lowered to assembly. To ensure fair evaluation, we primarily compare ITHICA checks against cpu-check's final output checks within the *same* ITHICA-compiled binary, rather than across independently compiled binaries (i.e., before and after ITHICA).

Coverage Gaps. We identify three main sources of limited coverage in our implementation: (1) atomic/volatile memory instructions (§1.1), (2) memory instructions in thread-unsafe code (§1) and (3) uninstrumented parts of `libc` and `libc++` libraries. For (2), even though ITHICA does not cover memory instructions of thread-unsafe multithreaded programs, it still covers most application-level instructions for a class of programs that are difficult to check with other techniques. For (3), by excluding the parts of `libc` and `libc++` not supported by `llvmlibc` and `llvmlibc++`, we leave kernel code executed as part of system calls unchecked, though ITHICA targets application-level code.

1.2 Additional Pedagogical Fault Injection

1.2.1 Simulating a Timing-Dependent Fault. We separately inject synthetic *setup* and *hold* time (timing-dependent) faults, using logical models of each developed in recent work [11]. Fig. 2 illustrates the simulated physical faults, both of which target the 32-bit `mult_result_q` output register of the CVA6 multiplier, whose input data is supplied by `mult_result_d`. For the setup fault, bit 31 of `mult_result_q` is overwritten with a logical 0 (instead of the data supplied on `mult_result_d`) on a rising clock edge only if the input signal `mult_result_d` differs from its value at the most recent prior clock edge. This condition is met for the first `MUL` instruction in our test, resulting in an incorrect output, but not for the second `MUL`, whose output is correct. For the hold fault, bit 31 of `mult_result_q` is overwritten with 0 on a rising clock edge only if `mult_result_d` differs from its value at the next clock edge. Here, the first `MUL` is unaffected, as it is followed by an instruction with the same `mult_result_d` value, while the second `MUL` produces an incorrect result.

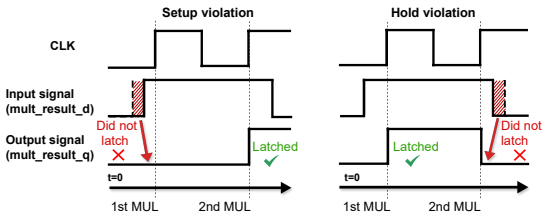

Figure 2: Physical fault model of single-bit setup and hold violations on the CVA6 multiplier output. In both cases, the two `MUL` instructions produce different results, allowing ITHICA to detect the fault.

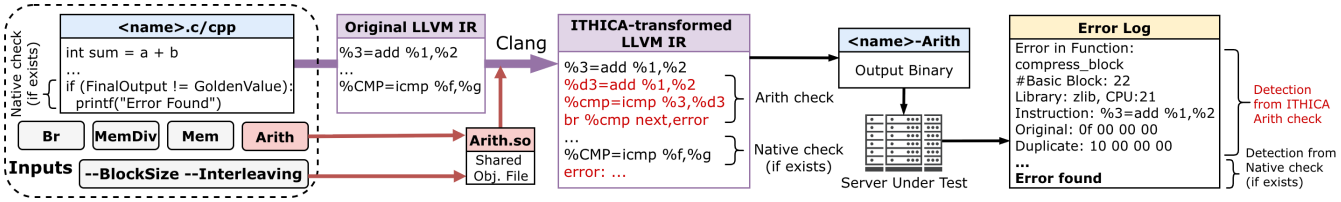

Figure 1: Given an input program (`<name>.cpp`), ITHICA applies one or more transformations—implemented in this paper for LLVM IR—configured with some `BlockSize` and `Interleaving`, and outputs a functional test (`<name>-ITHICA`).

Transform.	Target Instructions	Original Instr.	Inserted Validation Instructions	Inserted Check
`Arith`	Logic, binary, bitwise, vector, unary, aggregate & conversion ops, *getelementptr*, *icmp*, *fcmp* *select*, side-effect-free intrinsics & inline asm [9]	`r0 = op(s0,s1)`	`r1 = op(s0,s1)`	`eq(r0,r1)`
`Mem`	Load (non-atomic, non-volatile)	`v0 = load(a0)`	`v1 = load(a0)`	`eq(v0,v1)`
	Store (non-atomic, non-volatile)	`store(v0,a0)`	`v1 = load(a0)`	`eq(v0,v1)`
`MemDiv`	Load (non-atomic, non-volatile)	`v0 = load(a0)`	`v1 = load(a0); clflush(a0);` `v2 = load(a0)`	`eq(v0,v1) &&` `eq(v0,v2)`

	Store (non-atomic, non-volatile)	`store(v0,a0)`	`v1 = load(a0); mfence();`
			`v2 = load(a0); clflush(a0);`
			`v3 = load(a0)`
				`eq(v0,v1) &&`
				`eq(v0,v2) &&`
				`eq(v0, v3)`

`Br`	Conditional branches	`br(c0, t0,t1)`	`src: e = c0?t1:t0; store(e,tmp);`
			`t0: v0 = load(tmp);`
			`t1: v1 = load(tmp);`
				`src: None`
				`t0: eq(v0, t0)`
				`t1: eq(v1, t1)`

Table 1: Summary of ITHICA Transformations and Validation Rules

LLVM Instructions	`Arith`	`Mem` / `MemDiv`	`Br`	Excluded
Opcodes (# / %)	39 / 78.0%	2 / 4.0%	1 / 2.0%	8 / 16.0%
Static Instructions (PCs) (%)	62.3%	21.1%	1.4%	15.2%

Table 2: Coverage of different `ITHICA` passes for `CC` program.

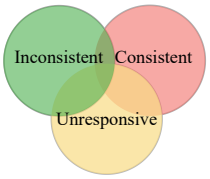

Figure 3: Classification of how hardware errors can manifest as three types of architectural errors [1, 10]. ITHICA explicitly detects pernicious inconsistent errors and implicitly detects unresponsive errors.

Technique	Check Type				Evaluation		Spec
	Golden Value	Cross-Core	Invert. Comp.	Intra-thr. Instruct.	Fault Inject.	Real HW	
`cpu-check` [3]	✗	✓	✓	✗	–	–	C/C++
`OpenDCDiag` [4]	✓	✓	✓	✗	✗	✓	C/C++
`SiliFuzz` [7]	✓	✓	✗	✗	–	–	ASM
`Harpocrates` [5]	✓	✓	✗	✗	✓	✗	Uarch
`ITHICA` (this paper)	✗	✗	✗	✓	✗	✓	LLVM

Table 3: Testing tools for defects compared by check type, evaluation platform, and required specification level.

1.3 Additional Related Work Comparison

Industrial Functional Tests for Defects. OpenDCDiag [4] is a framework-based suite of compute-intensive tests (e.g., MatMul), validated against precomputed golden values or across cores, and inverse computations (e.g., encryption-decryption). Cpu-check [3] is discussed in detail in the main paper. An orthogonal approach for test generation uses fuzzing to generate random instructions and expand short test cases [5–7], validating final outputs against golden values or across cores. Since defect manifestation appears sensitive to microarchitectural subtleties, the dependence of these approaches on CPU emulators and simulators may limit their effectiveness on real silicon. Reliance on below-ISA specifications for test generation [5, 6, 11] only accessible to chip manufacturers, renders these approaches inapplicable for chip users, including hyperscalars—ITHICA's focus. Nonetheless, ITHICA checks could be applied to fuzzer-generated sequences to provide automated error detection without relying on final outputs. We directly compare with Silifuzz, the only fuzzing technique publicly available, in the paper.

Consistent Errors. Although it is not possible to determine whether the six machines missed by ITHICA are due to the above coverage gaps or consistent errors, future work can apply functional diversity [2, 8, 12, 14] to target the latter.

References

[1] Saranyu Chattopadhyay, Keerthikumara Devarajegowda, Bihan Zhao, Florian Lonsing, Brandon A. D'Agostino, Ioanna Vavelidou, Vijay D. Bhatt, Sebastian Prebeck, Wolfgang Ecker, Caroline Trippel, Clark Barrett, and Subhasish Mitra. 2023. G-QED: Generalized QED Pre-silicon Verification beyond Non-Interfering Hardware Accelerators. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. https://doi.org/10.1109/DAC56929.2023.10247903

[2] Nikos Foutris, Dimitris Gizopoulos, Mihalis Psarakis, Xavier Vera, and Antonio Gonzalez. 2011. Accelerating microprocessor silicon validation by exposing ISA diversity. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture* (Porto Alegre, Brazil) *(MICRO-44)*. Association for Computing Machinery, New York, NY, USA, 386–397. https://doi.org/10.1145/2155620.2155666

[3] Google. 2020. Google cpu-check torture test. https://github.com/google/cpu-check.

[4] Intel. 2021. OpenDCDiag. https://github.com/opendcdiag.

[5] Nikos Karystinos, Odysseas Chatzopoulos, George-Marios Fragkoulis, George Papadimitriou, Dimitris Gizopoulos, and Sudhanva Gurumurthi. 2024. Harpocrates: Breaking the Silence of CPU Faults through Hardware-in-the-Loop Program Generation. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 516–531. https://doi.org/10.1109/ISCA59077.2024.00045

[6] Nikos Karystinos, George-Marios Fragkoulis, Odysseas Chatzopoulos, Dimitris Gizopoulos, and Sudhanva Gurumurthi. 2025. Harpocrates++: Automated Functional Program Generation against CPU Faults and Silent Data Corruptions. *IEEE Micro* (2025), 1–9. https://doi.org/10.1109/MM.2025.3640385

[7] Doug Kwan, Kostik Shtoyk, Kostya Serebryany, Maxim L Lifantsev, and Peter Hochschild. 2021. *SiliFuzz: Fuzzing CPUs by proxy.* Technical Report. Google.

[8] David Lin, Ted Hong, Yanjing Li, Farzan Fallah, Donald S Gardner, Nagib Hakim, and Subhasish Mitra. 2013. Overcoming post-silicon validation challenges through quick error detection (QED). In *2013 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 320–325.

[9] llvm-language-ref 2022. LLVM Language Reference Manual. https://llvm.org/docs/LangRef.html. Accessed: 2022-10-19.

[10] Florian Lonsing, Subhasish Mitra, and Clark W. Barrett. 2020. A Theoretical Framework for Symbolic Quick Error Detection. In *2020 Formal Methods in Computer Aided Design, FMCAD 2020, Haifa, Israel, September 21-24, 2020*. IEEE, 1–10. https://doi.org/10.34727/2020/ISBN.978-3-85448-042-6_9

[11] Jiacheng Ma, Majd Ganaiem, Madeline Burbage, Theo Gregersen, Rachel McAmis, Freddy Gabbay, and Baris Kasikci. 2025. Proactive Runtime Detection of Aging-Related Silent Data Corruptions: A Bottom-Up Approach. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4* (Hilton La Jolla Torrey Pines, La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 220–235. https://doi.org/10.1145/3622781.3674182

[12] N. Oh, S. Mitra, and E.J. McCluskey. 2002. ED4I: error detection by diverse data and duplicated instructions. *IEEE Trans. Comput.* 51, 2 (2002), 180–199. https://doi.org/10.1109/12.980007

[13] openssl-manual [n. d.]. OPENSSL Debian Manpages. https://manpages.debian.org/testing/libssl-doc/OPENSSL_LH_doall_arg.3ssl.en.html.

[14] Ilya Wagner and Valeria Bertacco. 2008. Reversi: Post-silicon validation system for modern microprocessors. In *2008 IEEE International Conference on Computer Design*. 307–314. https://doi.org/10.1109/ICCD.2008.4751878